ADMIN ACCESS, USER FLOWS & STANDARD OPERATING PROCEDURES

LiveSession Proctor — LiveKit online session platform with lightweight proctoring telemetry

PREPARED FOR

CPREC Nepal • livestream repository

CONTENTS

8 user flow diagrams • 8 step-by-step SOPs • admin login code patch • troubleshooting matrix

1. Executive summary

This document is the operational runbook for the LiveSession Proctor platform (repository **github.com/cprec-nepal/livestream**). It covers how an administrator logs into the monitoring dashboard, how students join live sessions, how proctoring telemetry is collected and surfaced, and how operators should respond to proctoring alerts. Each section pairs a flow diagram with a step-by-step Standard Operating Procedure (SOP) so the same information is usable both as a training aid and as an on-call reference.

A critical finding of this review is documented in section 2: **the current codebase has no admin authentication**. The /admin.html dashboard is openly accessible to anyone who can reach the URL. Section 3 contains a ready-to-apply code patch that adds server-side password authentication with a signed JWT cookie. Applying this patch is a prerequisite before deploying the platform to production.

Document scope

- **Section 2** — documents the missing authentication as it stands today.
- **Section 3** — provides a ready-to-apply code patch (server.js + admin.html + .env additions).
- **Section 4** — eight flow diagrams covering admin login, monitoring, alert handling, student join, token issuance, and the two telemetry channels.
- **Section 5** — eight Standard Operating Procedures, one per flow, written for both operators and engineers.
- **Appendices** — environment variable reference, troubleshooting matrix, alert type reference, and git deployment commands.

2. Critical gap: no admin authentication

> **CRITICAL.** In the codebase shipped to github.com/cprec-nepal/livestream, the admin dashboard at **/admin.html** has **no authentication**. Anyone who can reach the URL can see live participant lists, room names, tab-switch alerts, and proctoring telemetry. This must be fixed before the platform is used with real students.

2.1 What the code currently does

The Express server in server.js serves the contents of public/ unconditionally via express.static(...). The admin dashboard at /admin.html opens a Socket.io connection and immediately emits admin:join, which subscribes the socket to the admin-broadcast room. There is no middleware that verifies the requester is an administrator. The Socket.io server also does not verify credentials on connection.

2.2 What this means in practice

- A student who guesses or learns the URL /admin.html can monitor the entire cohort's tab switches, snapshot hashes, and identities.
- There is no audit trail of who viewed the dashboard or when. If an alert is missed, there is no way to determine whether an admin was actually watching.
- If the platform is exposed to the public internet (rather than an internal LAN), the dashboard is fully indexed and discoverable by search engines and scanners.
- Capacity enforcement in /api/token still works (the admin dashboard does not need it), but the absence of admin auth means a malicious user could call socket.emit("admin:join") from a browser console and start receiving admin events.

2.3 Recommended remediation

Apply the code patch in section 3. It adds a single-password login backed by an environment variable (ADMIN_PASSWORD) and a signed JWT cookie (ADMIN_JWT_SECRET). This is the minimum viable authentication for a single-admin deployment. For multi-admin deployments with audit trails, see section 3.4 (upgrade path).

3. Recommended fix: admin login code patch

This section contains three patches: a server-side patch that adds login/logout endpoints and an auth middleware, a frontend patch that adds a login screen and silent token refresh, and an environment variable addition. Apply them in this order. All line references are against the codebase as shipped to the repository on commit 3c7b92f (initial release).

3.1 Environment variables to add to .env

Append these two lines to your .env file (do NOT commit the real values; the .gitignore already excludes .env):

```
# Admin dashboard authentication
ADMIN_PASSWORD=replace-with-a-long-random-string
ADMIN_JWT_SECRET=replace-with-a-different-long-random-string
ADMIN_COOKIE_TTL_HOURS=8
```

Generate strong values with, for example, openssl rand -hex 32. The password should be at least 16 characters; the JWT secret at least 32 characters.

3.2 Patch to server.js

Apply this patch at the top of server.js, immediately after the existing require() block and after the LIVEKIT_URL constant. It loads jsonwebtoken, reads the new env vars, and registers three routes plus an auth middleware. Add the new dependency with npm install jsonwebtoken cookie-parser.

```
// === BEGIN ADMIN AUTH PATCH ===
// Add to the requires block at the top of server.js:
const jwt = require('jsonwebtoken');
const cookieParser = require('cookie-parser');

// Add after the LIVEKIT_URL constant:
const ADMIN_PASSWORD = process.env.ADMIN_PASSWORD || '';
const ADMIN_JWT_SECRET = process.env.ADMIN_JWT_SECRET || 'change-me';
const ADMIN_COOKIE_TTL_HOURS = parseInt(process.env.ADMIN_COOKIE_TTL_HOURS || '8', 10);

// Fail-fast in production if ADMIN_PASSWORD is unset
if (NODE_ENV === 'production' && (!ADMIN_PASSWORD || ADMIN_PASSWORD.length < 12)) {
  console.error('[FATAL] ADMIN_PASSWORD must be set to at least 12 chars in production.');
  process.exit(1);
}

// Add after `app.use(express.json(...))`:
app.use(cookieParser());

// Constant-time string comparison (defends against timing attacks)
function safeEqual(a, b) {
  const ba = Buffer.from(String(a));
  const bb = Buffer.from(String(b));
  if (ba.length !== bb.length) return false;
  return crypto.timingSafeEqual(ba, bb);
}

// Auth middleware for /admin* paths
function requireAdmin(req, res, next) {
  const token = req.cookies && req.cookies.admin_session;
```

```
  if (!token) return res.redirect('/admin/login');
  try {
    jwt.verify(token, ADMIN_JWT_SECRET, { algorithms: ['HS256'] });
    return next();
  } catch (err) {
    res.clearCookie('admin_session');
    return res.redirect('/admin/login');
  }
}

// GET /admin/login  — render the login form
app.get('/admin/login', (req, res) => {
  const errMsg = req.query.err ? 'Invalid password' : '';
  res.type('html').send(
    '' +
    'Admin login' +
    'body{font-family:system-ui;background:#0b0f1a;color:#e7ecf5;' +
    'display:grid;place-items:center;min-height:100vh;margin:0}' +
    'form{background:#111726;padding:32px;border-radius:14px;' +
    'border:1px solid rgba(255,255,255,.08);width:320px}' +
    'h1{margin:0 0 16px;font-size:18px}' +
    'input{width:100%;padding:12px;border-radius:8px;border:1px solid #1a2235;' +
    'background:#0b0f1a;color:#fff;font-size:14px;margin-bottom:12px;box-sizing:border-box}' +
    'button{width:100%;padding:12px;border-radius:8px;border:0;' +
    'background:linear-gradient(135deg,#3b82f6,#6f5cff);color:#fff;font-weight:600;cursor:pointer}' +
    '.err{color:#ff5a6a;font-size:12px;margin-bottom:8px}' +
    '' +
    'Admin login' + errMsg +
    '' +
    'Sign in' +
    ''
  );
});

// POST /admin/login  — verify password, set cookie
app.post('/admin/login', (req, res) => {
  const submitted = (req.body && req.body.password) || '';
  if (!safeEqual(submitted, ADMIN_PASSWORD)) {
    console.warn('[admin] failed login attempt from ' + req.ip);
    return res.redirect('/admin/login?err=1');
  }
  const token = jwt.sign({ role: 'admin' }, ADMIN_JWT_SECRET, {
    algorithm: 'HS256',
    expiresIn: ADMIN_COOKIE_TTL_HOURS * 3600,
  });
  res.cookie('admin_session', token, {
    httpOnly: true,
    secure: NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: ADMIN_COOKIE_TTL_HOURS * 3600 * 1000,
  });
  return res.redirect('/admin.html');
});

// POST /admin/logout  — clear cookie
app.post('/admin/logout', (req, res) => {
  res.clearCookie('admin_session');
  return res.redirect('/admin/login');
});

// Protect /admin.html and /admin/* (place BEFORE express.static)
app.use('/admin.html', requireAdmin);
app.use('/admin', requireAdmin);

// Socket.io: also verify the cookie on connection
```

```
io.use((socket, next) => {
  const cookie = socket.handshake.headers.cookie || '';
  const m = cookie.match(/admin_session=([^;]+)/);
  if (!m) return next(new Error('unauthorized'));
  try {
    jwt.verify(m[1], ADMIN_JWT_SECRET, { algorithms: ['HS256'] });
    return next();
  } catch (err) {
    return next(new Error('unauthorized'));
  }
});
// === END ADMIN AUTH PATCH ===
```

WARNING. The io.use() middleware at the bottom of the patch is critical: without it, a malicious user could open a Socket.io connection directly to /socket.io/ and emit admin:join from a console, bypassing the page-level guard.

3.3 Patch to public/admin.html

Add a "Sign out" button to the top bar. Find the existing top bar header (around line 260 in the current file) and add the highlighted button just before the closing </header> tag:

```
   Sign out
```

No other changes are needed on the frontend. The browser carries the cookie automatically; if the cookie is missing or expired, the auth middleware redirects to /admin/login.

3.4 Upgrade path (multi-admin, audit trail)

The single-password approach is appropriate for a single admin or a small team that shares one password. When you need per-administrator accountability, replace the env-var password with a small JSON user store:

- **users.json** — an array of { username, passwordHash (bcrypt), displayName, role } entries, stored outside the web root.
- **Login form** — extend the form to ask for username + password. The POST handler bcrypt-compares the password hash.
- **JWT payload** — include { sub: username, name: displayName, role } so the admin UI can show who is currently signed in.
- **Audit log** — append every login (success or failure) and every alert acknowledgment to audit.log with timestamp + IP + username.

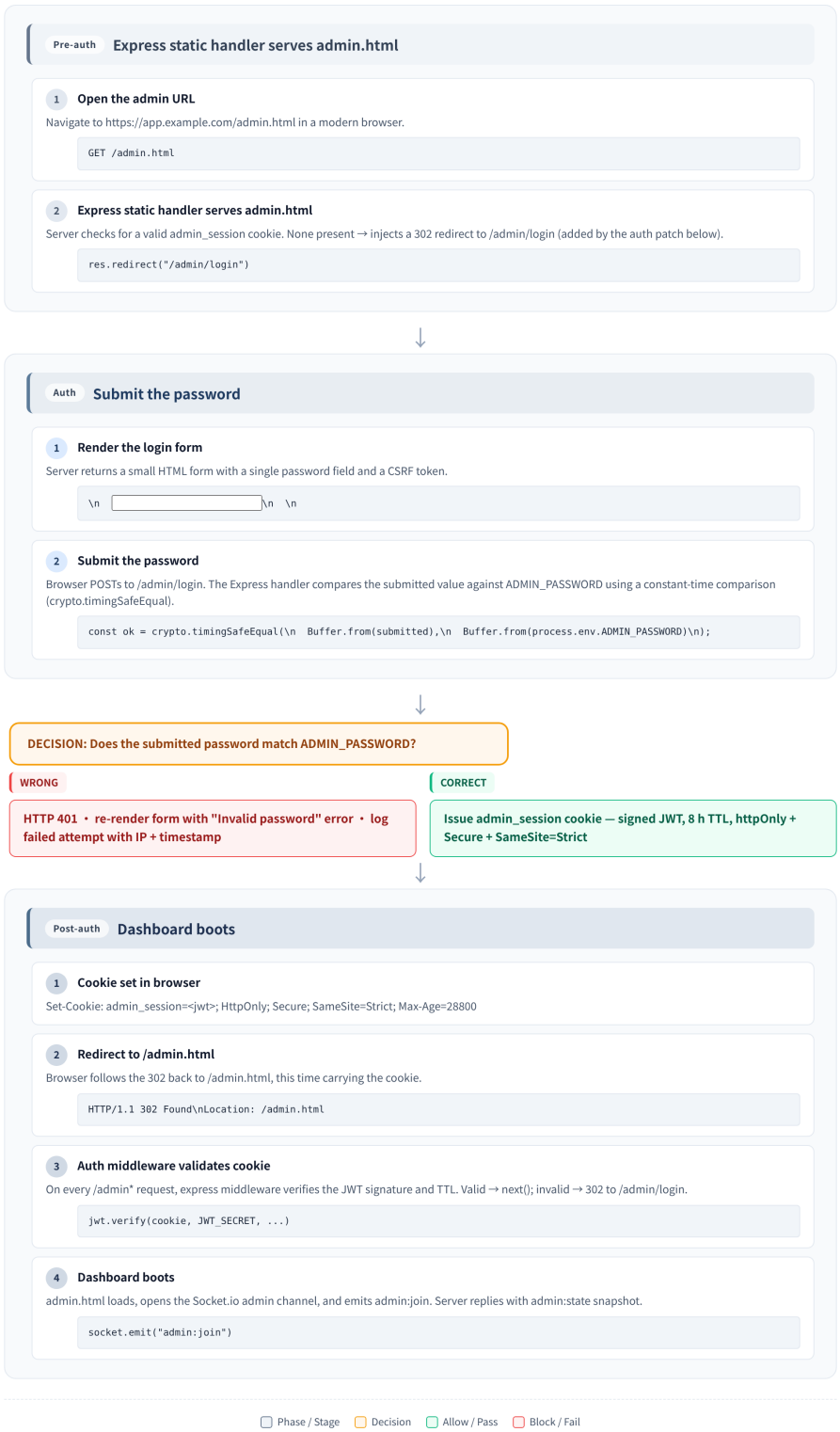
4. User flow diagrams

The eight diagrams below show the end-to-end flow of every significant operation in the platform. Each diagram uses the same phased-vertical layout: phases (the colored cards) represent major stages of the flow; numbered steps inside each phase are sequential actions; amber diamonds are decision points; green boxes are "allow / pass" outcomes; red boxes are "block / fail" outcomes.

Flow 1 — Admin login

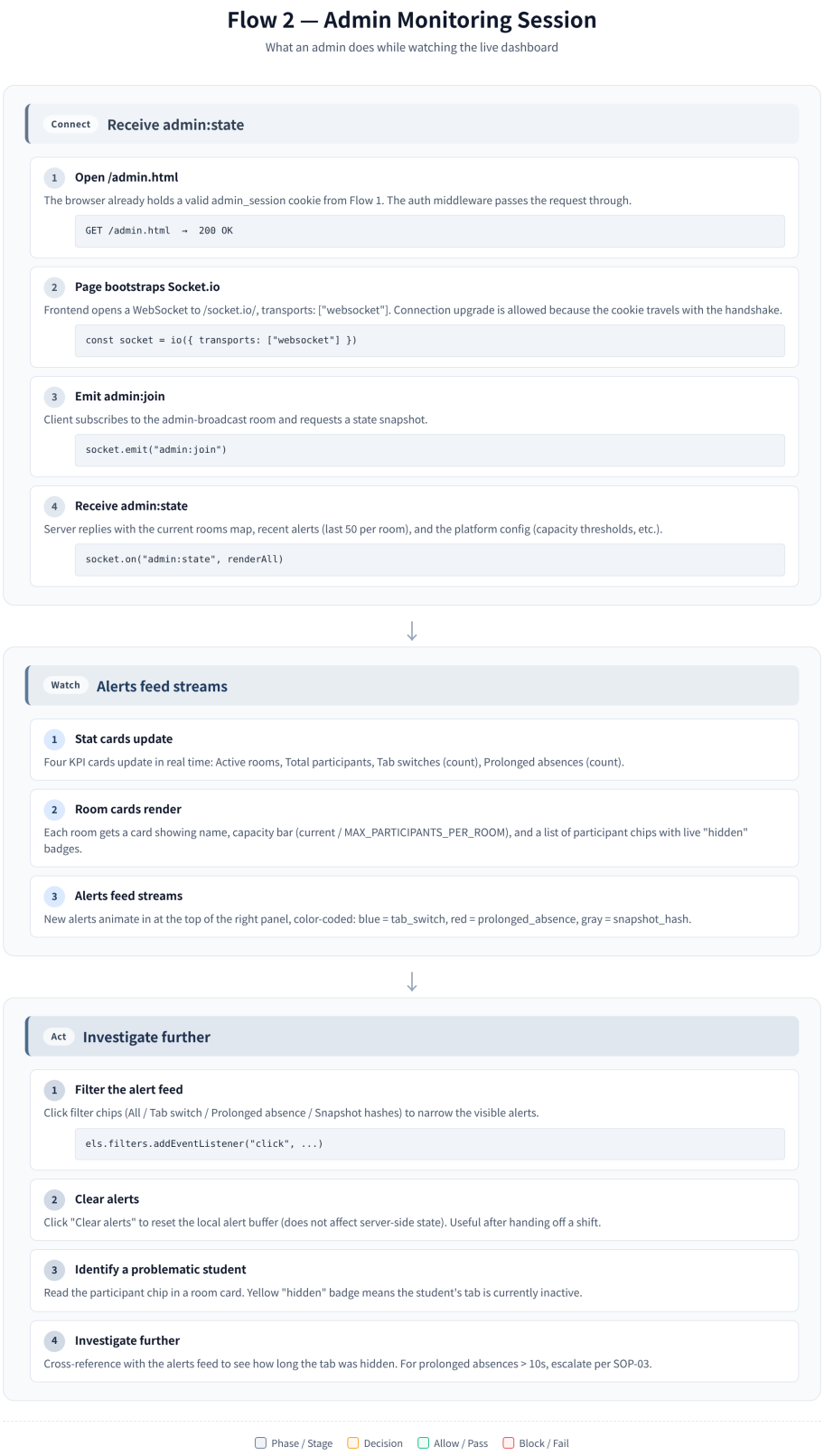
Flow 1 — Admin Login

How an administrator authenticates and reaches the live monitoring dashboard



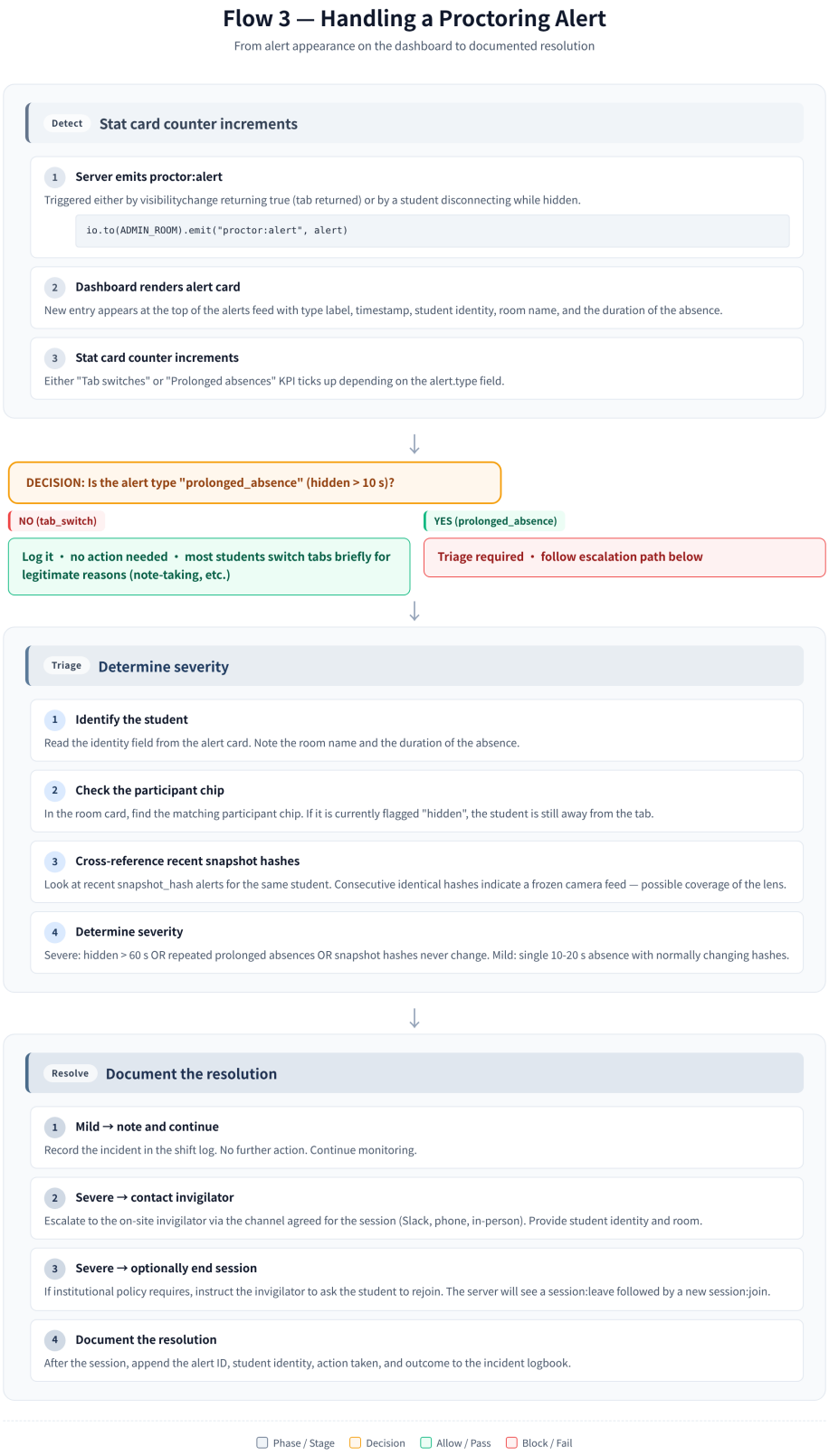
How an administrator authenticates and reaches the live monitoring dashboard. This diagram reflects the post-patch state described in section 3.

Flow 2 — Admin monitoring session



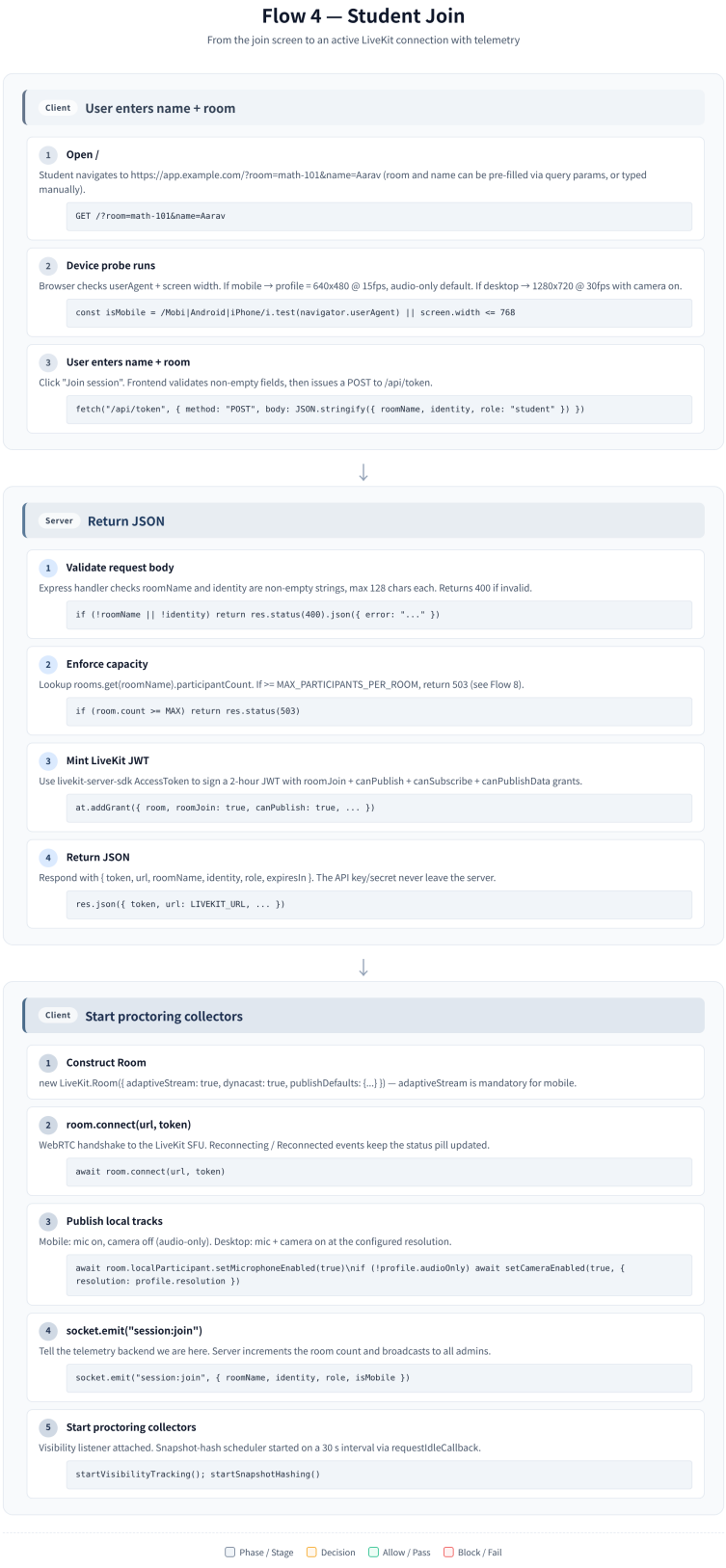
What an admin does while watching the live dashboard: connect, monitor, act.

Flow 3 — Handling a proctoring alert



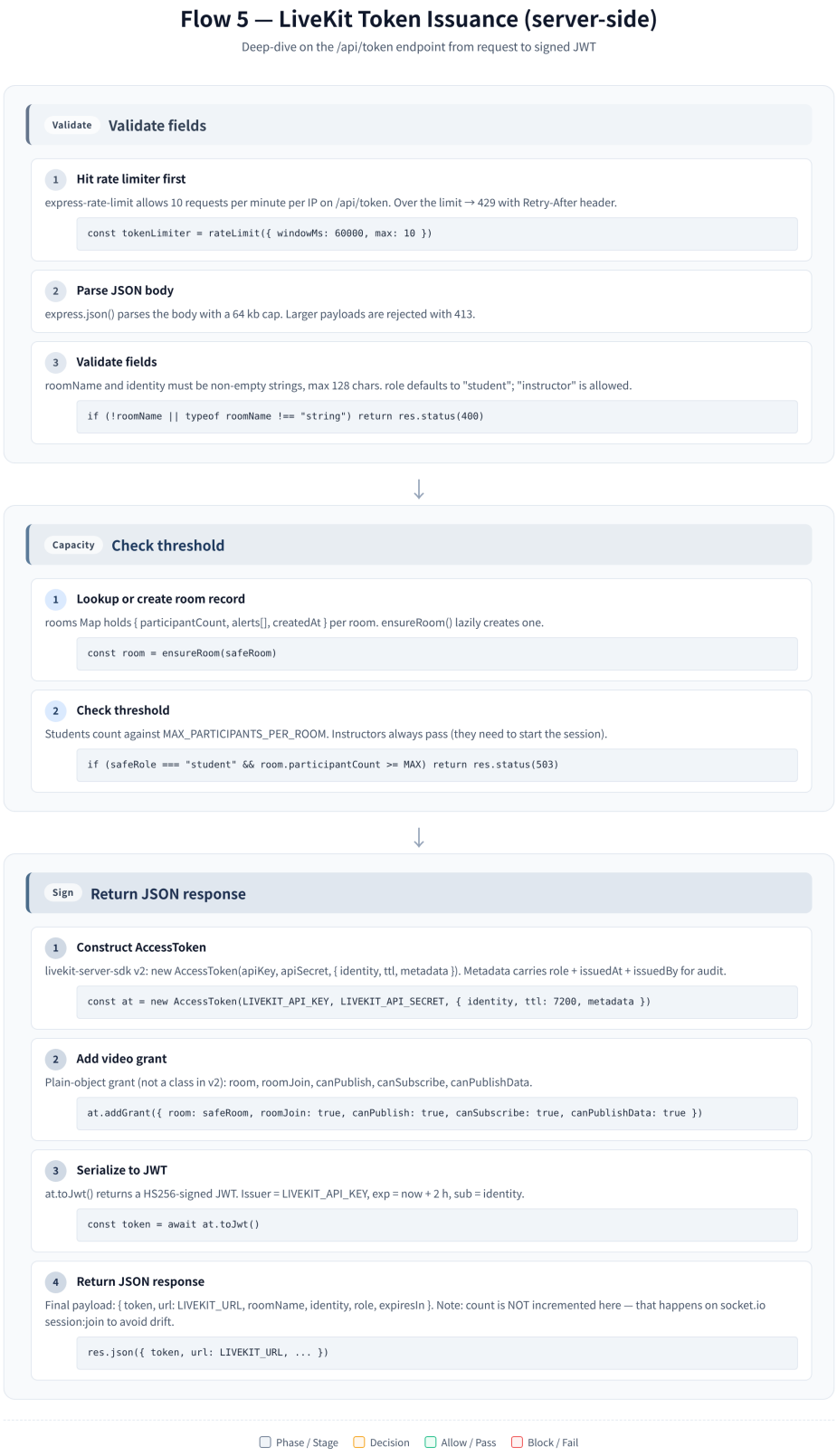
From alert appearance on the dashboard to documented resolution. Includes the triage decision tree for prolonged absences.

Flow 4 — Student join



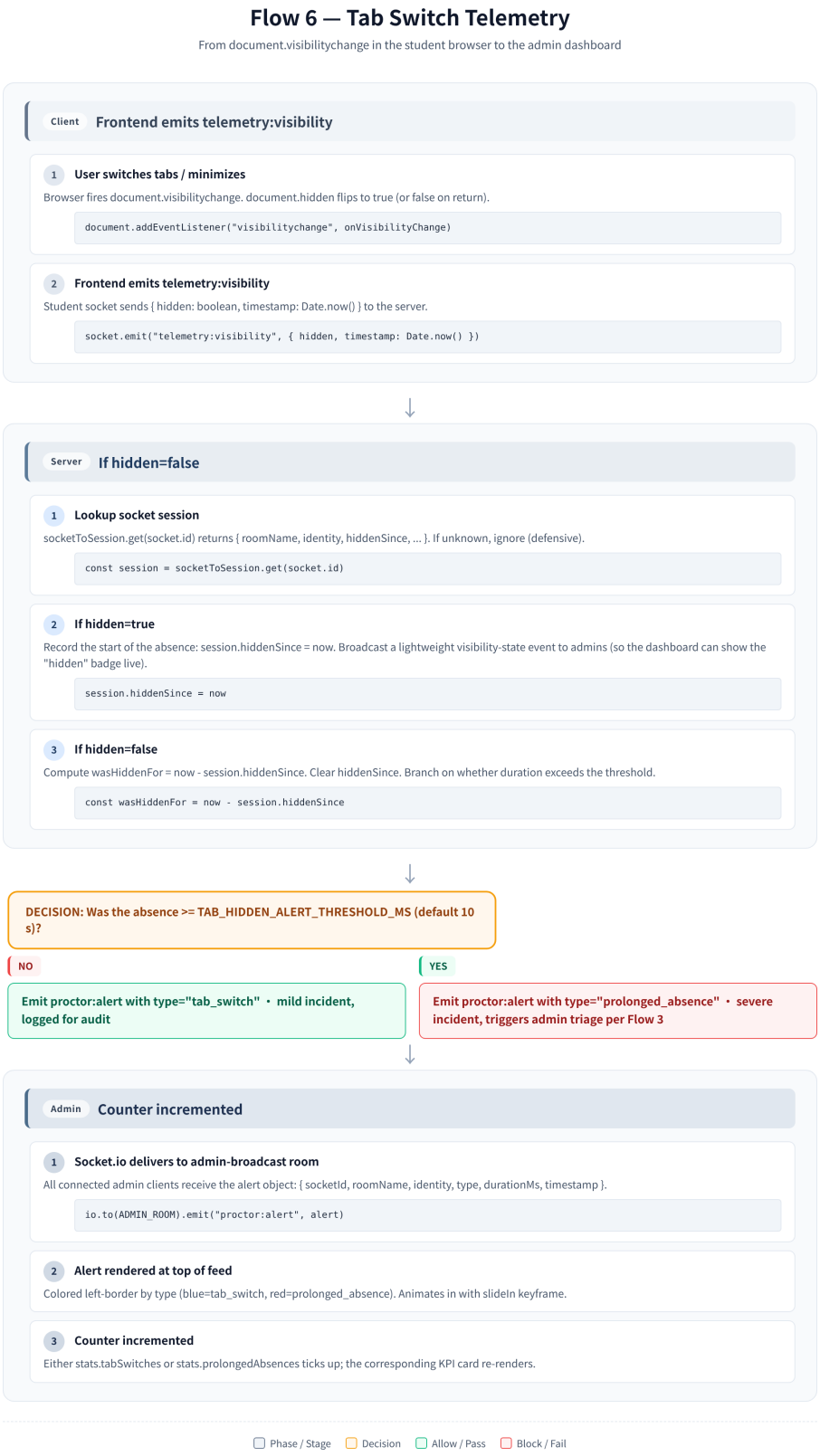
From the join screen to an active LiveKit connection with telemetry collectors running. Cross-references Flow 5 (token issuance) and Flow 7 (snapshot hashing).

Flow 5 — LiveKit token issuance (server-side)



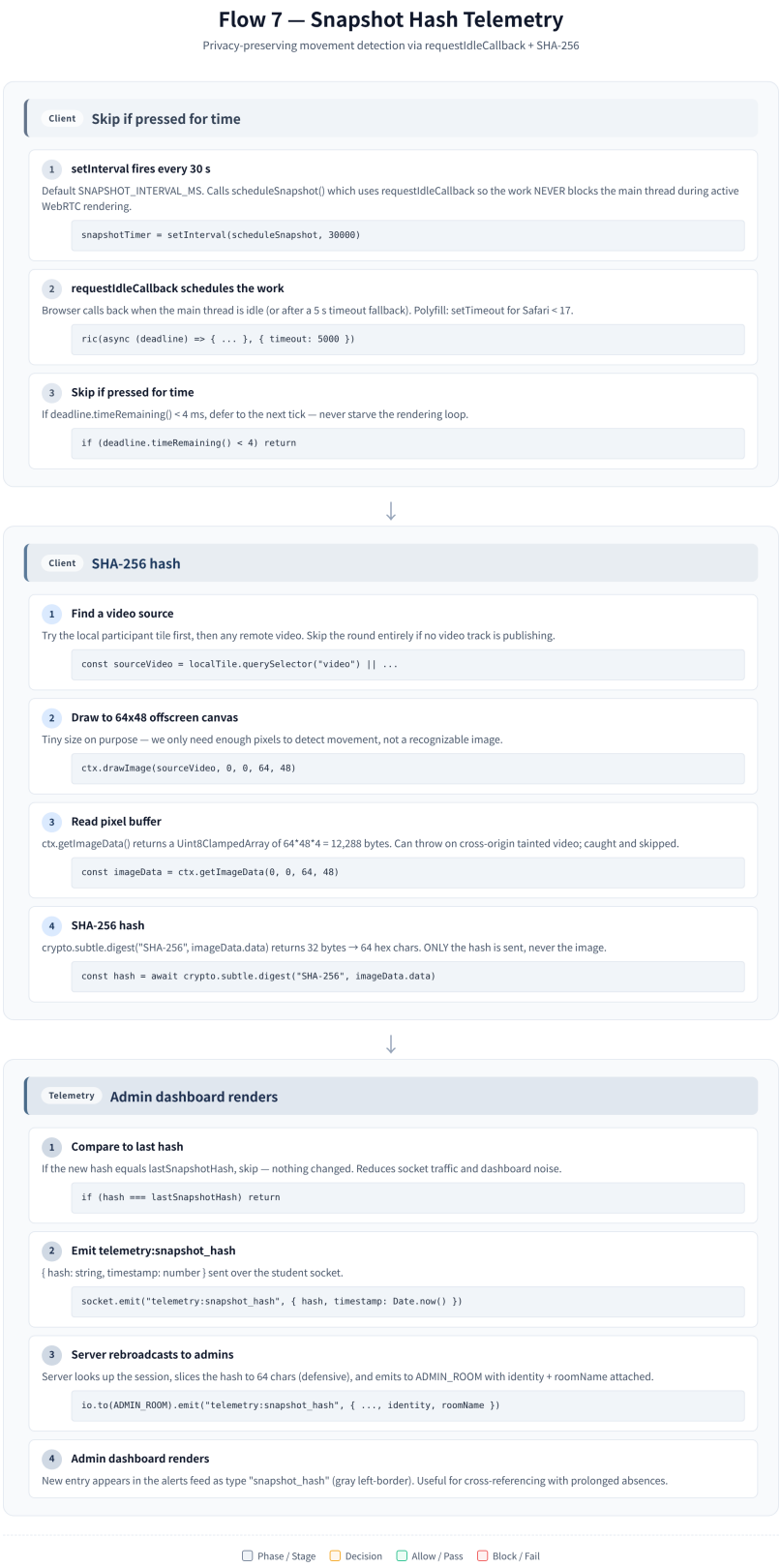
Deep-dive on the /api/token endpoint: validation, capacity check, JWT signing. The server-side half of Flow 4.

Flow 6 — Tab switch telemetry



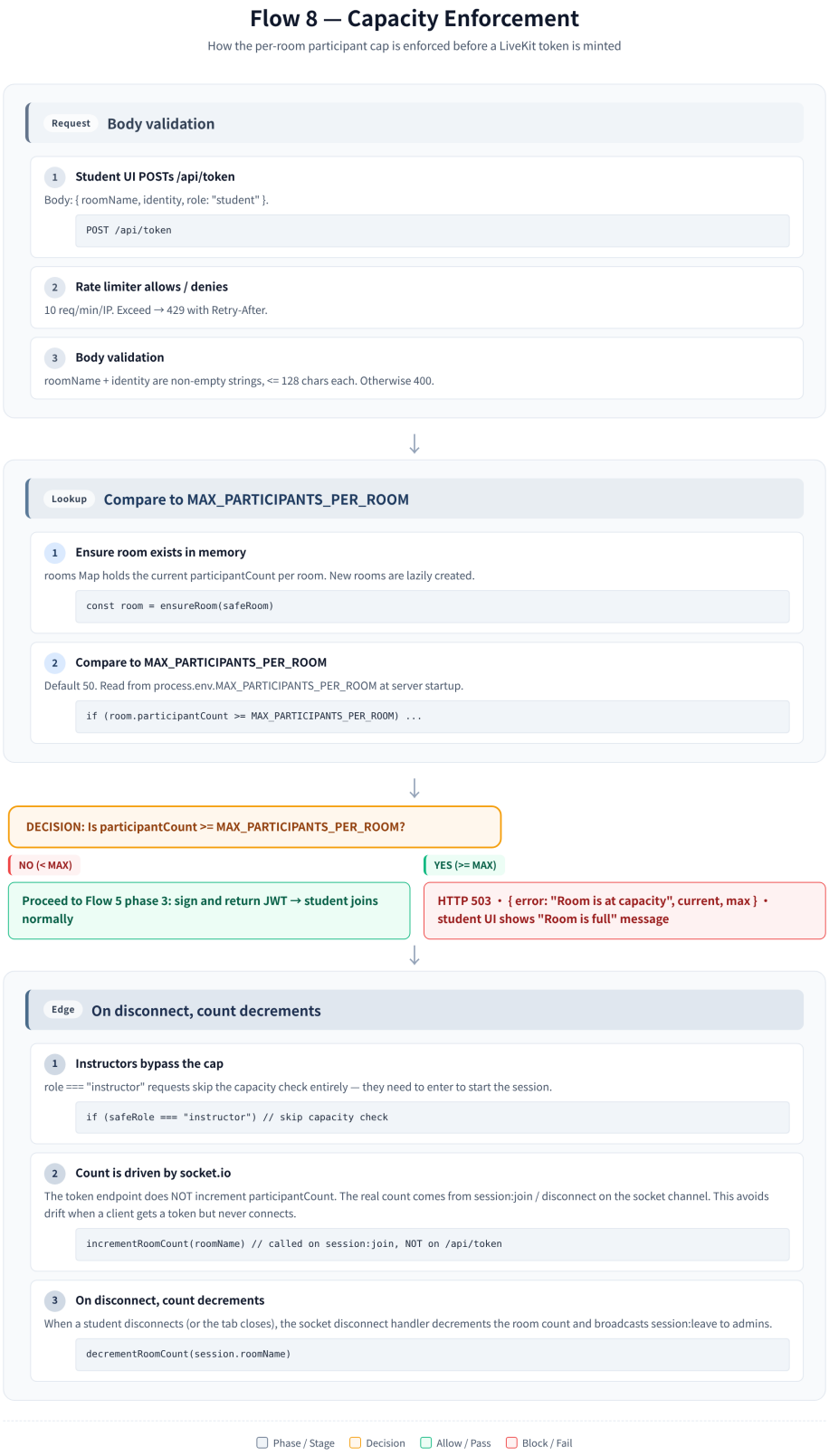
From document.visibilitychange in the student browser to the admin dashboard alert card.

Flow 7 — Snapshot hash telemetry



Privacy-preserving movement detection via requestIdleCallback + SHA-256. Only the hash is sent, never the image.

Flow 8 — Capacity enforcement



How the per-room participant cap is enforced before a LiveKit token is minted. Includes the instructor override and the socket-driven count semantics.

5. Standard Operating Procedures

Each SOP below corresponds to one of the eight flow diagrams in section 4. SOPs are written for a mixed audience: an operator on shift (typically a session administrator or invigilator) and the engineer on call. Where a step requires elevated privileges, the required role is noted in italics.

SOP-01 — Admin login

Trigger: An administrator needs to access the monitoring dashboard.
Prerequisites: The server is running; the administrator knows the ADMIN_PASSWORD; the administrator has a modern browser; the device is on a network that can reach the server.

Procedure:

1. Open the admin URL in a browser: https://app.example.com/admin.html. The browser is redirected to /admin/login because no admin_session cookie is present.
2. On the login screen, type the ADMIN_PASSWORD into the password field and click **Sign in**.
3. On success, the server issues an admin_session cookie (HttpOnly, Secure, SameSite=Strict, 8 h TTL) and redirects to /admin.html.
4. On failure (HTTP 401), the login screen re-renders with an "Invalid password" message. The server logs the failed attempt with the source IP. After three failures from the same IP within five minutes, treat as a suspected scanning attempt and review the access log.
5. Once the dashboard loads, verify the green Connected pill in the top-right corner. The four KPI cards should display zero or non-zero counts depending on whether sessions are already in progress.

Expected outcome: The admin dashboard is fully loaded, the connection pill is green, and the rooms list / alerts feed are receiving live updates.

> **WARNING.** If the login screen loops back to itself even after a correct password, check that ADMIN_JWT_SECRET is set and consistent across restarts. A rotated secret invalidates all previously-issued cookies.

SOP-02 — Admin monitoring session

Trigger: A live session is in progress (or about to start) and an admin needs to monitor it.
Prerequisites: SOP-01 completed; at least one student is expected to join a room.

Procedure:

1. Confirm the dashboard shows the expected room in the Active rooms panel. If the room does not appear within 60 s of the scheduled start time, contact the instructor.
2. Watch the four KPI cards: Active rooms, Total participants, Tab switches, Prolonged absences. A healthy session has tab switches in the low single digits per minute and zero prolonged absences.

3 For each room card, check the capacity bar. If it is amber (>= 80 % of MAX_PARTICIPANTS_PER_ROOM), prepare to refuse additional joiners. If it is red (100 %), students attempting to join will see a "Room is at capacity" message; inform the instructor.

4 In the room card's participant chips, watch for yellow hidden badges. A brief hidden state is normal (the student may be reading a question paper in another tab). A persistent hidden state with no recovery is suspicious.

5 Use the alert filter chips (All / Tab switch / Prolonged absence / Snapshot hashes) to focus on one category at a time when the feed gets busy.

6 At the end of your shift, click **Clear alerts** only if you are handing off to another admin and have already documented incidents in the shift log. Clearing alerts does not affect server-side state.

Expected outcome: The admin maintains continuous awareness of all active rooms, can identify potential incidents within 30 s of their occurrence, and can hand off to the next shift with a clean dashboard.

SOP-03 — Handling a proctoring alert

Trigger: A new alert appears in the Proctoring alerts feed.
Prerequisites: SOP-01 completed; the admin is actively watching the dashboard.

Procedure:

1 Read the alert type label (top-left of the alert card): TAB SWITCH (blue), PROLONGED ABSENCE (red), or SNAPSHOT HASH (gray).

2 For TAB SWITCH: no action required. The student switched away briefly and returned. Note the duration in the shift log only if it exceeds 5 s.

3 For PROLONGED ABSENCE: read the duration field. If >= 60 s, escalate to the on-site invigilator immediately (per the escalation matrix in Appendix B). If 10-60 s, monitor the same student for the next 5 min and escalate only if repeated.

4 For SNAPSHOT HASH: compare the hash to the previous hash for the same student (look up the prior SNAPSHOT HASH alert). If the hash has not changed across 3+ consecutive snapshots, the camera may be covered or frozen; escalate to the invigilator.

5 When escalating, give the invigilator the student identity and room name from the alert card. Do not send the raw hash or telemetry payload over insecure channels.

6 After the invigilator resolves the issue (the student returns to the tab, uncovers the camera, or is removed from the session), document the incident in the shift log: timestamp, identity, alert type, duration, action taken, outcome.

Expected outcome: Every alert is triaged within 30 s of arrival, severe alerts are escalated within 60 s, and all incidents are documented before end of shift.

SOP-04 — Student join

Trigger: A student wants to join a live session.
Prerequisites: The student has the session URL and their name; the instructor has already started the room on LiveKit.

Procedure:

1. Open the URL provided by the instructor: https://app.example.com/?room=math-101&name;=Aarav. The room and name fields are pre-filled from the URL.
2. If the URL did not include query parameters, type the room name and your name into the form on the join screen.
3. Click **Join session**. The browser sends a POST to /api/token. If the room is at capacity, the UI shows "Room is at capacity" and the join attempt fails; contact the instructor.
4. On success, the browser receives a LiveKit JWT and connects to the LiveKit server. The status pill in the top-right turns green ("Live").
5. Grant camera and microphone permissions when the browser prompts. On mobile, only the microphone is requested by default; click ☒ **Enable camera** in the controls bar to publish video.
6. Your local tile (labeled "You") appears in the video grid. Other participants' tiles appear as they join or publish tracks.
7. Throughout the session, keep the tab focused. Tab switches are logged and may trigger proctoring alerts (see SOP-03).
8. To leave, click **Leave**. The browser disconnects from LiveKit and from the telemetry socket.

Expected outcome: Student is connected to the LiveKit room with their microphone (and optionally camera) publishing; telemetry is flowing to the admin dashboard.

SOP-05 — LiveKit token issuance (operator perspective)

Trigger: A student reports that the join button shows an error and they cannot enter the room.
Prerequisites: Operator/engineer role with SSH access to the application server.

Procedure:

1. SSH to the application server. Tail the application log: journalctl -u livestream -f (or pm2 logs livestream if running under pm2).
2. Reproduce the join attempt. Look for a [token] log entry showing the request body and the response code.
3. If the response is 400, the client is sending an invalid body. Verify the student UI is up to date (force-refresh the browser).
4. If the response is 429, the client has hit the rate limiter (10 requests / minute / IP). Wait one minute and retry.
5. If the response is 503, the room is at capacity. Either increase MAX_PARTICIPANTS_PER_ROOM in .env and restart the server, or ask the instructor to split the cohort.

6 If the response is 500, the LiveKit SDK failed to mint the token. Check LIVEKIT_API_KEY and LIVEKIT_API_SECRET in .env; verify they match the LiveKit server's config.yaml.

7 After resolving the issue, the student should retry the join. No server restart is needed unless .env was changed.

Expected outcome: The /api/token endpoint returns 200 with a valid JWT, and the student can join.

SOP-06 — Tab switch telemetry (operator verification)

Trigger: The admin dashboard is not showing tab-switch alerts even though students are visibly switching tabs.
Prerequisites: Engineer role with browser devtools access; the student UI loaded correctly.

Procedure:

1 In the student's browser, open devtools → Console. Switch the browser tab away and back. Look for a console log line from the visibility listener.

2 In devtools → Network → WS, find the socket.io connection. Confirm a telemetry:visibility event is sent each time the tab visibility changes.

3 On the server, tail the application log. The [io] log entries should show the socket id and the event being received.

4 On the admin dashboard, verify that a telemetry:visibility event updates the participant chip's hidden badge in real time.

5 When the tab returns to focus, verify a proctor:alert is emitted (visible in the alerts feed) with the correct duration. If the duration is >= 10 s, the alert type should be prolonged_absence (red), otherwise tab_switch (blue).

6 If no events are reaching the server, check that the Socket.io connection is in the websocket transport (not polling) and that no reverse proxy is stripping the WebSocket upgrade headers.

Expected outcome: Tab switches produce a proctor:alert within 1 s of the tab returning to focus; the alert type correctly reflects the absence duration.

SOP-07 — Snapshot hash telemetry (operator verification)

Trigger: The admin dashboard is not showing snapshot_hash alerts, OR a student's hashes never change.
Prerequisites: Engineer role; student has granted camera permission.

Procedure:

1 In the student's browser devtools → Console, wait 30 s (the default snapshot interval). Look for the "Snapshot hash sent" log line.

2 If no log line appears, check the console for errors. The most common cause is a tainted canvas (cross-origin video); in that case the snapshot is silently skipped, which is expected behavior.

3 If the log line appears but no alert shows on the admin dashboard, verify the telemetry:snapshot_hash event is being sent over the socket (devtools → Network → WS → messages).

4 On the admin dashboard, verify the Snapshot hashes filter chip is selected. If the feed has many entries, switch the filter to isolate them.

5 If the same student's hashes are identical across 3+ snapshots, ask the student to wave or move. If hashes still do not change, the camera may be covered or frozen; escalate per SOP-03.

6 If hashes are missing for a mobile student who joined audio-only, that is expected: there is no video source to hash. The telemetry stream is simply silent for that student.

Expected outcome: Snapshot hashes appear every 30 s per student (per video source), with hashes changing as the camera content changes.

SOP-08 — Capacity enforcement (operator runbook)

Trigger: A student reports "Room is at capacity" while the instructor says there are fewer students than expected.
Prerequisites: Engineer role with SSH access to the application server.

Procedure:

1 Check the dashboard: read the capacity bar for the affected room. Compare the displayed count to the actual number of students the instructor reports.

2 If the dashboard count is higher than the instructor's count, there are likely stale sessions (sockets that disconnected without the server noticing, e.g. on a network failure). Restart the application server to clear the in-memory rooms map.

3 If the dashboard count is correct but the cap is genuinely too low for the cohort, raise MAX_PARTICIPANTS_PER_ROOM in .env and restart the server. Coordinate with LiveKit capacity planning: each participant publishes ~1.5 Mbps up and subscribes to N-1 other participants, so check the LiveKit server's outbound bandwidth.

4 If the dashboard count is lower than expected and students still see "Room is at capacity", check the application log for errors during /api/token. A 503 response includes the current and max fields in the JSON body — inspect them in the browser devtools.

5 After the fix, verify by having a test student join successfully. Confirm the dashboard count increments by one when their session:join arrives.

Expected outcome: The capacity count matches reality, students can join when room count is below the cap, and joiners are correctly refused when the cap is reached.

A. Appendix A — Environment variables reference

Variable	Purpose	Default	Status
LIVEKIT_API_KEY	LiveKit API key	YOUR_LIVEKIT_API_KEY	Required
LIVEKIT_API_SECRET	LiveKit API secret (used to sign JWTs)	YOUR_LIVEKIT_API_SECRET	Required
LIVEKIT_URL	LiveKit server WebSocket URL	wss://your-livekit-server.example.com	Required
PORT	HTTP port the app server listens on	3000	Optional
NODE_ENV	Node environment; production enables fail-fast checks	production	Recommended
CORS_ORIGIN	Comma-separated allowed origins, or * for all	*	Set to real origins in prod
MAX_PARTICIPANTS_PER_ROOM	Hard cap before token issuance is refused	50	Optional
TAB_HIDDEN_ALERT_THRESHOLD_MS	Time a tab can stay hidden before an elevated alert fires	10000	Optional
SNAPSHOT_INTERVAL_MS	Interval between client snapshot-hash reports	30000	Optional
ADMIN_PASSWORD	Admin dashboard password (added by the auth patch in section 3)	unset	Required after patch
ADMIN_JWT_SECRET	Secret used to sign admin session JWTs	unset	Required after patch
ADMIN_COOKIE_TTL_HOURS	Admin session cookie TTL in hours	8	Optional

B. Appendix B — Troubleshooting matrix

Symptom	Likely cause	Resolution
Student cannot join; "Connecting..." stuck	LiveKit server unreachable / wrong URL	Verify LIVEKIT_URL in .env; check LiveKit server is running; check TLS cert
Student cannot join; "Room is at capacity"	Room participant count >= MAX_PARTICIPANTS_PER_ROOM	Raise cap in .env and restart, OR move students to a new room
Student cannot join; "Invalid password"	Student mistyped name OR server log shows 400	Verify roomName field is correct; refresh student UI
Admin dashboard shows "Disconnected" pill	Socket.io disconnected; possibly auth middleware rejected the socket	Verify admin_session cookie is present; re-login if expired
Admin login loops back to login screen	ADMIN_JWT_SECRET changed since cookie was issued	Clear cookies and re-login; ensure ADMIN_JWT_SECRET is stable across restarts
No alerts appearing despite visible tab switches	Student UI did not load socket.io.js OR visibility listener not attached	Check browser console for errors; verify /socket.io/socket.io.js returns 200
Snapshot hashes never change for a student	Camera covered, frozen, or video track not publishing	Ask student to wave; check local tile shows video; escalate per SOP-03
Server fails to boot in production	Placeholder LIVEKIT_API_KEY or ADMIN_PASSWORD	Set real values in .env; restart server
Token endpoint returns 500	LiveKit SDK failed to sign JWT	Verify LIVEKIT_API_KEY/SECRET match LiveKit server config.yaml
Capacity count drifts from reality	Stale socket sessions after network failures	Restart application server to clear in-memory rooms map
Mobile student reports poor video quality	adaptiveStream not enabled or bandwidth too low	Confirm adaptiveStream:true in room opts (it is on by default); student can disable camera and use audio-only
Admin dashboard UI is blank / unstyled	CSP blocking CDN scripts	Verify Content-Security-Policy allows cdn.jsdelivr.net for script-src

C. Appendix C — Alert type reference

Alert type	UI color	Meaning	Operator action
tab_switch	Blue	Student switched away from the session tab for less than 10 seconds.	None required. Note in shift log if duration > 5 s.
prolonged_absence	Red	Student switched away from the session tab for 10 seconds or longer. Also fired on disconnect-while-hidden.	Triage per SOP-03. Escalate if duration > 60 s or repeats.
snapshot_hash	Gray	SHA-256 hash of the student's current video frame, sent every 30 seconds via requestIdleCallback.	Cross-reference with prior hashes. Identical hashes across 3+ snapshots suggest a frozen/covered camera.

Session lifecycle events (not alerts, but appear in the rooms panel)

Event	Meaning
session:join	A new socket connected and emitted session:join. Participant count incremented.
session:leave	A socket disconnected. Participant count decremented. Room card removed after 5 s if count reaches zero.
telemetry:visibility	Live state update (no alert). Updates the participant chip "hidden" badge in real time.
admin:state	Initial snapshot sent on admin:join. Contains current rooms + last 50 alerts per room.

D. Appendix D — Git deployment commands

From a clean checkout of the repository, applying the section 3 patch and pushing the updated code:

```
# 1. Clone the repo (or pull if you already have it)
git clone https://github.com/cprec-nepal/livestream.git
cd livestream

# 2. Install the new dependency required by the auth patch
npm install jsonwebtoken cookie-parser

# 3. Apply the patch from section 3 (server.js, public/admin.html, .env)
#    Edit the files manually or apply via `git apply admin-auth.patch`

# 4. Configure credentials locally (do NOT commit the real .env)
cp .env.example .env
nano .env   # set LIVEKIT_API_KEY, LIVEKIT_API_SECRET, LIVEKIT_URL,
            # ADMIN_PASSWORD, ADMIN_JWT_SECRET

# 5. Smoke-test in development mode
NODE_ENV=development npm start
# In another terminal:
curl -s http://localhost:3000/health
curl -sI http://localhost:3000/admin.html   # should be 302 to /admin/login

# 6. Commit the patch
git add server.js public/admin.html .env.example package.json package-lock.json
git commit -m "feat(admin): add password + JWT-cookie auth for admin dashboard"

# 7. Push to GitHub
git push origin main

# 8. On the production server: pull, install, restart
git pull origin main
npm install --omit=dev
sudo systemctl restart livestream   # or: pm2 restart livestream
```

OK. After deployment, verify by visiting /admin.html in a browser. You should be redirected to /admin/login. Sign in with ADMIN_PASSWORD. If you can sign in and see the dashboard, the patch is live.